

CSIT 112 Lab 5: Parallelizing Image Processing (Box Blur Filter)

Introduction: How a Blur Filter Works

A blur filter is one of the simplest and most common image processing techniques. Its goal is to smooth out details and reduce noise by averaging the color values of neighboring pixels. The effect makes edges softer and the image less sharp.

Blurring works by sliding a small matrix (called a kernel or filter) across the image. For each pixel, the algorithm looks at its surrounding neighborhood (e.g., a 3×3 window), averages their color values, and replaces the central pixel with this average.

A 3×3 box blur kernel looks like this:

$\frac{1}{9} * \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

Because each pixel can be processed independently, blur filters are highly parallelizable—perfect for exploring multithreading and synchronization in image processing.

Learning Goals

- Understand data-parallel workloads on images.
- Practice Java concurrency using `ExecutorService` and `CountDownLatch`.
- Compare sequential and parallel executions.
- Learn about thread safety and synchronization.

Prerequisites

- JDK 8+ installed
- Folders: `images/input/` and `images/output/`
- Example input image: `aroma.jpg`
- Reference: `EdgeDetectionSeq`, `EdgeDetectionParallel`, `EdgeDetectionParallelSync`

What You'll Build

Implement a 3×3 box blur filter three ways:

1. Sequential version (provided)
2. Parallel version (no synchronization)
3. Parallel version (with synchronization)

Starter Code

Students receive these starter files:

- BoxBlurSeq.java – complete sequential version
- BoxBlurParallel.java – skeleton with TODOs (no sync)
- BoxBlurParallelSync.java – skeleton with TODOs (with sync)

Lab Steps

Part A — Sequential Baseline

1. Compile and run **BoxBlurSeq.java**.
2. Note the output time and image.

Question: Why skip the 1-pixel border? (Hint: to avoid out-of-bounds access.)

Part B — Parallelization Without Synchronization

Modify **BoxBlurParallel.java** to:

- Use ExecutorService with a fixed thread pool.
- Assign one task per image row.
- Do not synchronize setRGB calls.

Part C — Parallelization With Synchronization

Modify **BoxBlurParallelSync.java** to wrap setRGB() in synchronized(outputImage).

Experiments

1. Vary thread count (1, 2, 4, 8, 16, 32) and record execution times.
2. Try different chunk sizes (multiple rows per task).
3. Compare output images for correctness.

Deliverables

- Three output images:
 - image_blur_seq.jpg
 - image_blur_parallel.jpg
 - image_blur_parallel_sync.jpg
- 1–2 page report with performance measurements, graphs, and discussion. The template to the report is attached.

Grading Rubric (50 pts)

- Correct blur implementation – 15 pts
- Working parallel versions (with/without sync) – 20 pts
- Performance experiments & discussion – 5 pts
- Clear report & analysis – 10 pts

Instructor Notes

- The no-sync version may appear fine due to pixel independence but is not guaranteed safe.
- For better performance, use per-thread row buffers or batched writes.

Lab 5 Report Template

Lab Title: Parallelizing Image Processing — Box Blur Filter

Student Name: _____

Student ID: _____

Date: _____

Course / Section: _____

1. Introduction

Briefly explain the purpose of this lab and what a blur filter does.

- What is the goal of blurring?
- How does the 3×3 box blur kernel work?
- Why is this a good example for parallelization?

(Write your paragraph here)

2. Implementation Overview

Describe what you implemented. Mention the three program versions:

- Sequential version (BoxBlurSeq.java)
- Parallel version without synchronization (BoxBlurParallel.java)
- Parallel version with synchronization (BoxBlurParallelSync.java)

Also summarize:

- How tasks were divided (e.g., one task per row)
- What synchronization method you used (if any)
- Any challenges you faced.

(Write your paragraph here)

3. Experimental Setup

Describe your test environment and configuration:

- CPU model and number of cores
- Java version
- Image size(s) used
- Thread counts tested (e.g., 1, 2, 4, 8, 16, 32)
- How you measured execution time

(Write your paragraph here)

4. Results and Performance Analysis

4.1. Execution Time

Version	1 Thread	2 Threads	4 Threads	8 Threads	16 Threads	32 Threads
Sequential		X	X	X	X	X
Parallel (no sync)						
Parallel (with sync)						

(Fill in actual timing results in milliseconds.)

4.2. Output Comparison

Describe and compare the output images:

- Were there any visual differences between the sequential and parallel versions?
- Did you observe any artifacts or noise in the no-sync version?
- Did synchronization fix those issues?

(Write your paragraph here)

4.3. Speedup and Efficiency

Calculate the speedup for each parallel version using:

$$\text{Speedup} = T_{\text{sequential}} / T_{\text{parallel}}$$

Discuss how efficiency changes with more threads.

(Write your analysis here)

5. Conclusion

Summarize the key findings of your lab:

- What did you learn about parallel programming and synchronization?
- How effective is parallelization for image processing tasks?
- What improvements or alternative designs would you suggest?

(Write your paragraph here)